Hochschule Konstanz
Fakultät Informatik

Software Architecture

REST

Prof. Dr. Marko Boger

Distributed Communication

Within one Language/System

- Sockets
- RMI
- EJB

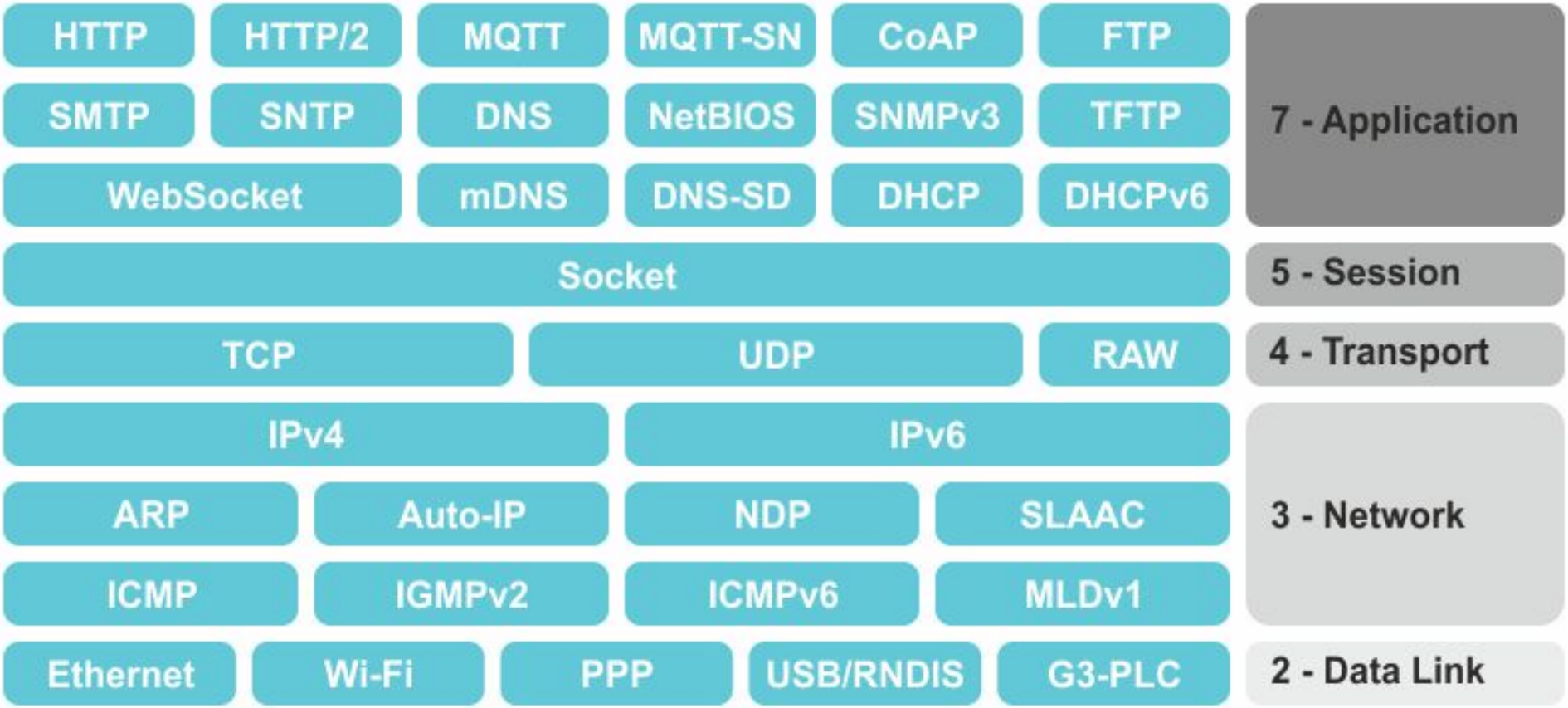
Between different Languages/Systems

- CORBA
- SOAP
- REST

Sockets

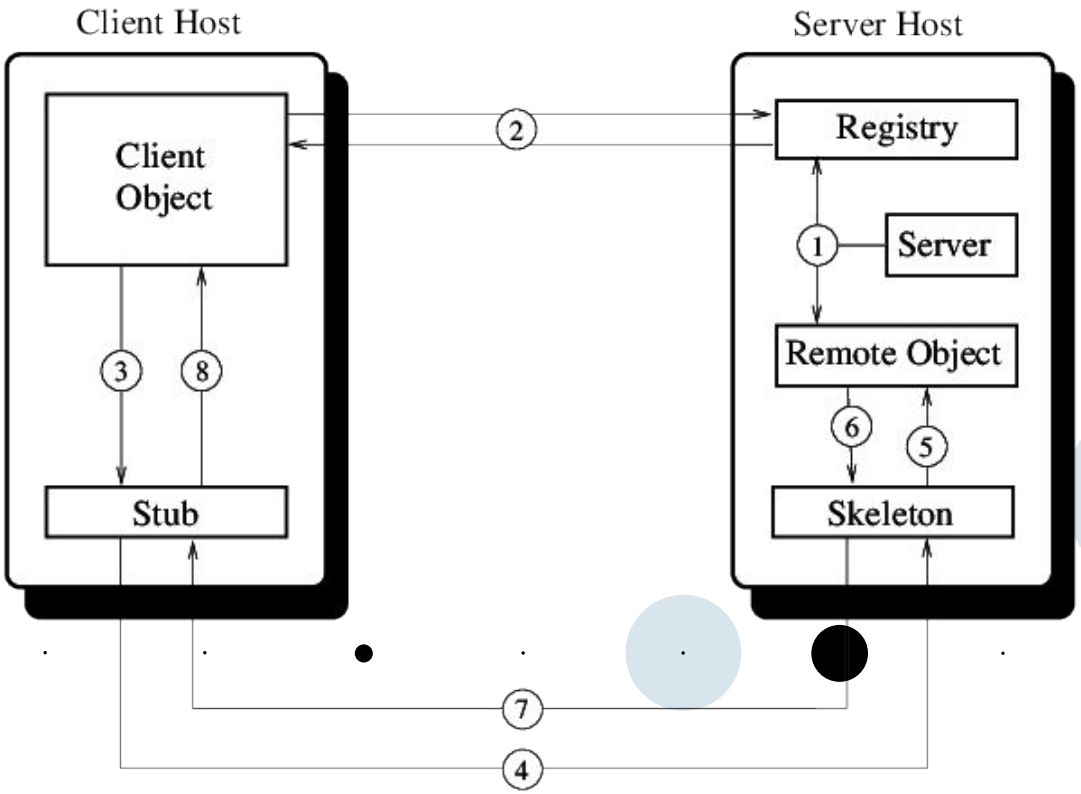
- A socket is an endpoint of a bidirectional inter-process communication flow using
  - TCP, provides reliable, ordered delivery of a stream of bytes or
  - UDP, provides an unreliable service and datagrams may arrive out of order, appear duplicated, or go missing without notice
- A socket address is the combination of
  - an IP address
    - the location of the computer
  - and a port
    - mapped to the application program process

Sockets in the OSI Model



RMI

- Remote Method Invocation
  - Built-in mechanism of Java for Client-Server communication
  - Normal call semantics of a method call
    - plus setup
    - plus exception handling



JavaBeans

JavaBeans is a component model for Java, initially used for Widgets in GUI Builders.

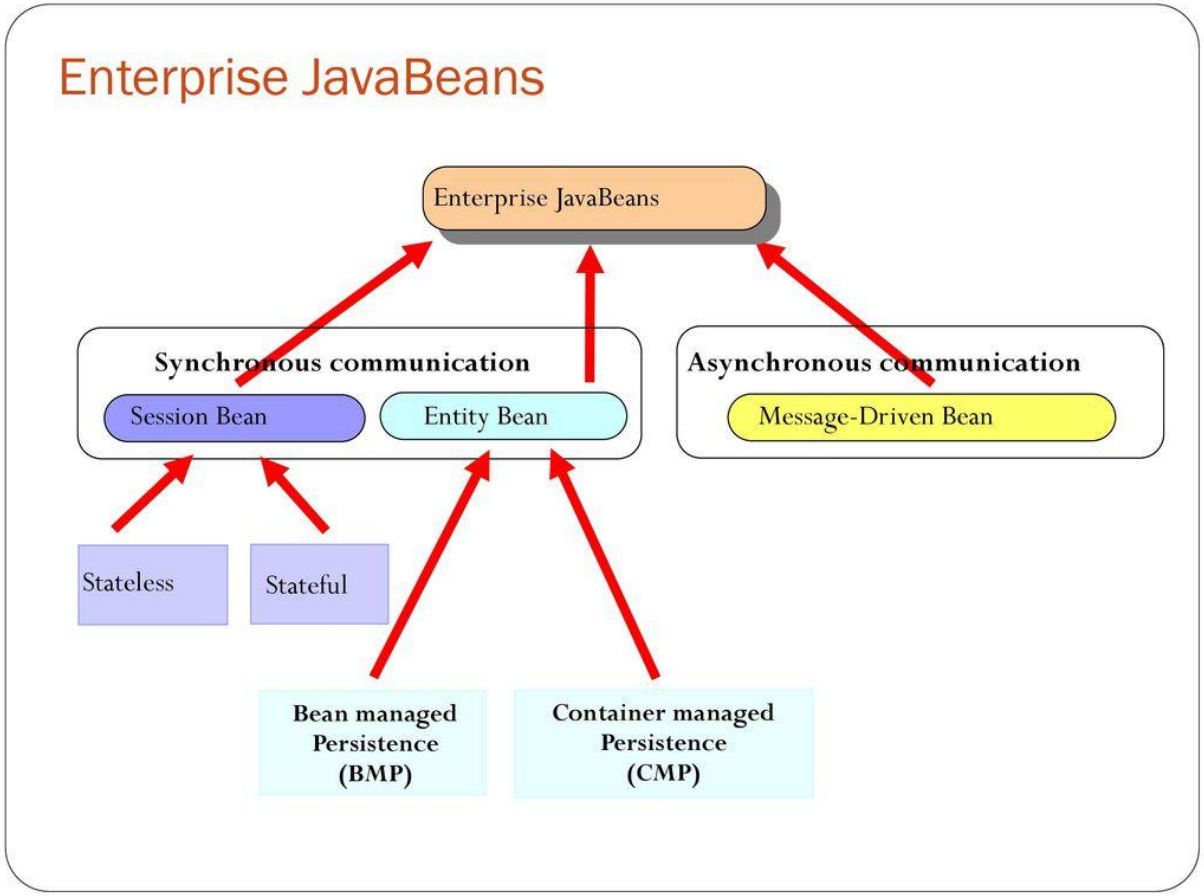
JavaBeans are classes that are serializable, have a zero-argument constructor, and allow access to properties using getter and setter methods.

Enterprise JavaBeans

An Enterprise JavaBeans (EJB) is a server-side software component that encapsulates business logic of an application.

An EJB web container provides a runtime environment for web related software components.

EJB Vocabulary

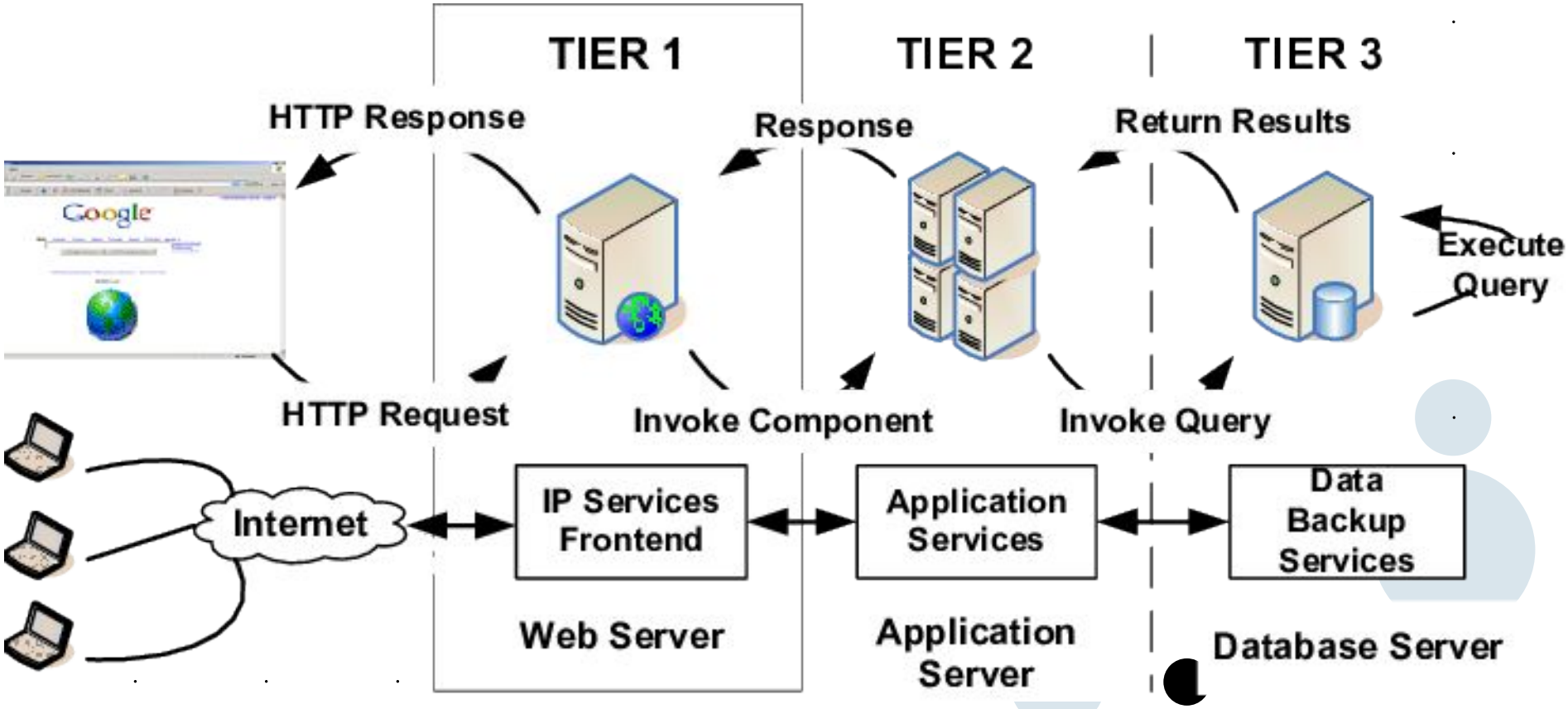


Application Server

EJB are deployed to Application Servers, like WebLogic, WebSpere, Glassfish.

The Application Server serves as a container to manage routine tasks like transaction management, security and scaling.

Three Tier Architecture



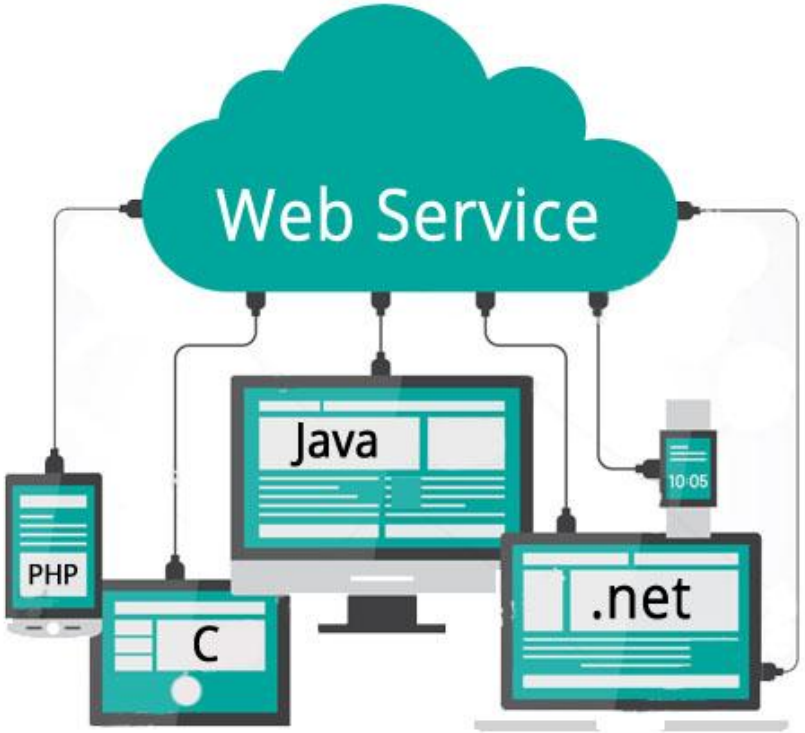
Limits of RMI

- RMI works well in closed distributed contexts, like a corporate network
  - Fixed to Java
  - Object-oriented
  - not Service-oriented
- It works well for connections on low API level
- But for
  - Loose coupling
  - Between separated corporations
  - With different languages
  - On a service level

We need something different

Web Services

A Web Service provides access to machine readable data or digital services, usually in XML or JSON format.

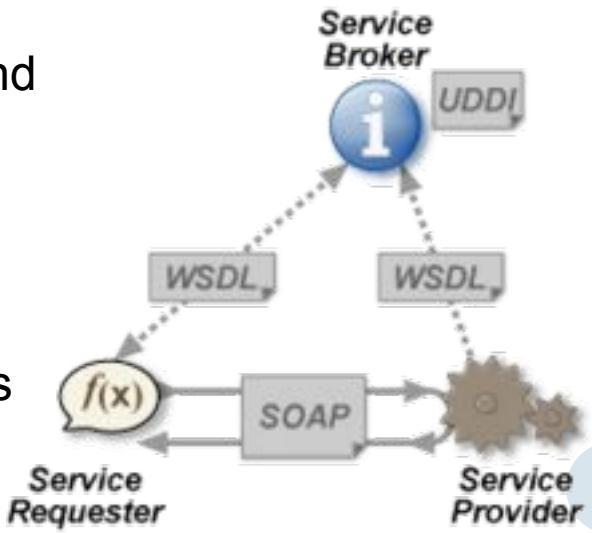


Early Web Service Technologies

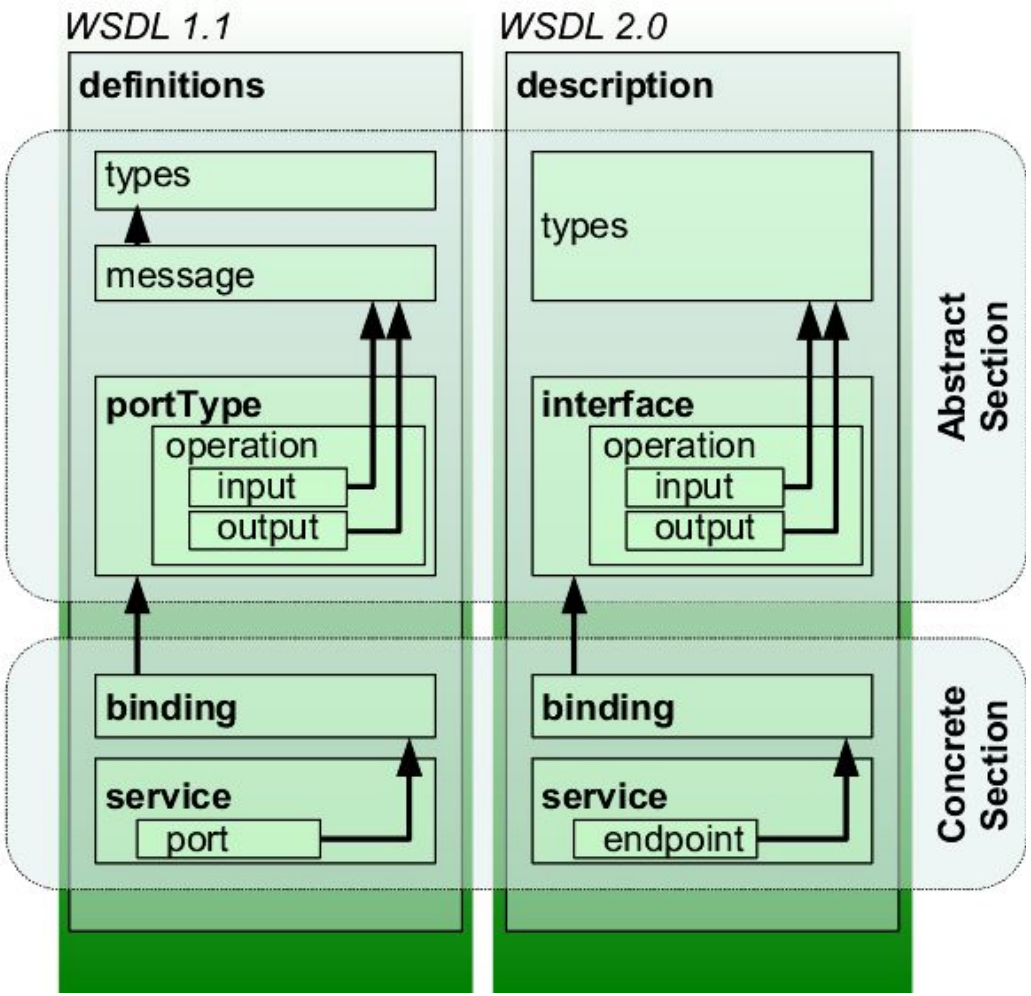
UDDI is an XML-based standard for describing, publishing, and finding web services. UDDI stands for Universal Description, Discovery, and Integration.

WSDL is an XML-based interface description language that is used for describing the functionality offered by a web service. It stands for Web Service Description Language.

SOAP is a messaging protocol specification for exchanging structured information in the implementation of web services. It stands for Simple Object Access Protocol.

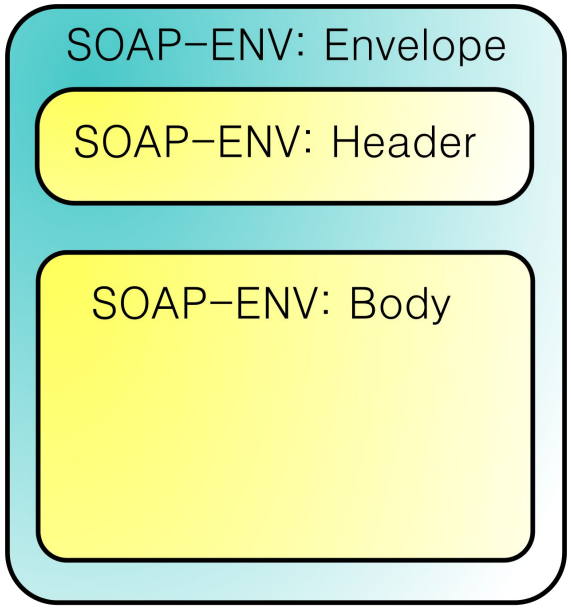


The Format of a WSDL file




```
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
  <xsd:complexType name="GridDataType">
    <xsd:sequence>
      <xsd:element name="gridData" type="xsd:string"></xsd:elemen
    </xsd:sequence>
  </xsd:complexType>
  <xsd:simpleType name="GridResultType">
    <xsd:restriction base="xsd:string"></xsd:restriction>
  </xsd:simpleType>
 </xsd:schema>
</wsdl:types>
<wsdl:message name="createRequest">
 <wsdl:part element="tns:create" name="parameters"/>
</wsdl:message>
<wsdl:message name="createResponse">
 <wsdl:part element="tns:createResponse" name="parameters"/>
</wsdl:message>
<wsdl:portType name="SudokuCreatePort">
 <wsdl:operation name="create">
  <wsdl:input message="tns:createRequest"/>
  <wsdl:output message="tns:createResponse"/>
 </wsdl:operation>
</wsdl:portType>
<wsdl:binding name="SudokuCreateSOAP" type="tns:SudokuCreatePort">
 <soap:binding style="document" transport="http://schemas.xmlsoap.o
 <wsdl:operation name="create">
  <soap:operation soapAction="http://www.eclipse.org/swordfish/samp
  <wsdl:input>
   <soap:body use="literal"/>
  </wsdl:input>
  <wsdl:output>
   <soap:body use="literal"/>
  </wsdl:output>
 </wsdl:operation>
</wsdl:binding>
<wsdl:service name="SudokuCreateService">
 <wsdl:port binding="tns:SudokuCreateSOAP" name="SudokuCreateSOAP">
  <soap:address location="http://localhost:8198/SudokuCreateServic
 </wsdl:port>
</wsdl:service>
</wsdl:definitions>
```

SOAP

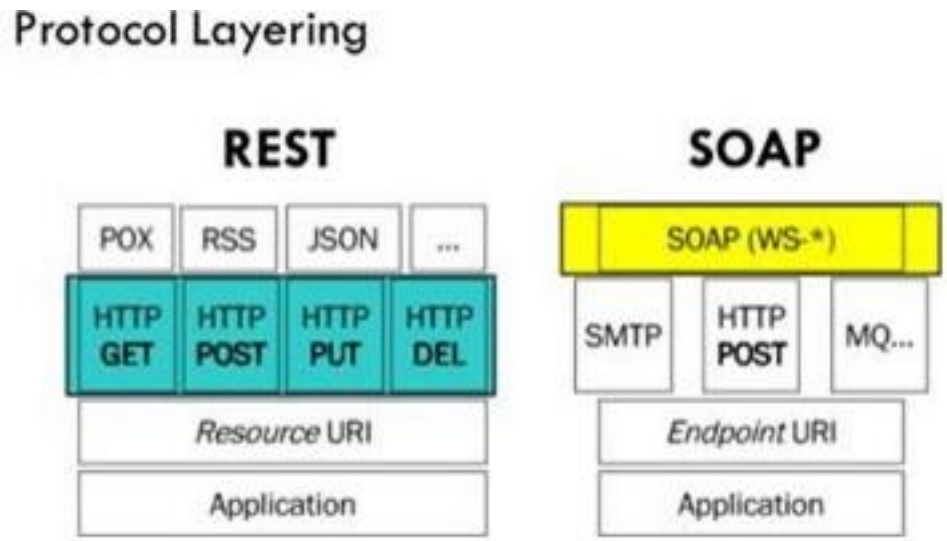



```
POST /InStock HTTP/1.1
Host: www.example.org
Content-Type: application/soap+xml; charset=utf-8
Content-Length: 299
SOAPAction: "http://www.w3.org/2003/05/soap-envelope"

<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
xmlns:m="http://www.example.org">
  <soap:Header>
  </soap:Header>
  <soap:Body>
    <m:GetStockPrice>
      <m:StockName>GOOG</m:StockName>
    </m:GetStockPrice>
  </soap:Body>
</soap:Envelope>
```

REST

- REST is an architecture style for web application APIs
- REST stands for REpresentational State Transfer
- It was described in a PhD-Thesis by Roy Fielding in 2000
  - He also helped develop the HTTP protocol, central to REST



REST vs CRUD

- REST maps requests to the verbs of HTTP
- It maps nicely to CRUD
  - POST - CREATE
  - GET - READ
  - PUT - UPDATE
  - DELETE - DELETE

Client-Server

- Clients
  - Clients initiate requests by directly using the HTTP protocol
  - The client addresses objects on the server using URIs
- Servers
  - Servers typically respond to requests with XML or JSON message
    - But could also send HTML, CSV or other formats
  - Servers do not know the state of the client

REST in Play

- Play makes it very easy to develop REST APIs
  - The routes file maps URIs to API calls
  - The template mechanism makes it easy to create responses

- The routes file

```
GET     /user/{id}          Application.user
POST    /user/              Application.createUser
PUT     /user/{id}          Application.updateUser
DELETE  /user/{id}          Application.deleteUser
```

Some Guidelines for URLs

- Use two URLs per resource type
  - One for the collection, one for instances
- Use nouns, not verbs for the URIs
- Use plural for the collection, singular for the instance
- Put everything else behind the ?

Akka HTTP

- Akka HTTP is a library to handle HTTP directly
  - on the client as well as
  - on the server side
- Akka HTTP provides high and low level APIs to handle HTTP in a very powerful way
  - Backchannel
- For us, only the high level API is of interest
  - Routing DSL

Routing DSL in Akka HTTP

- Akka HTTP provides a very nice DSL to express the routing table in code
- We can define routings per
  - URL or
  - HTTP verb

```
val route =
      path("hello") {
        get {

complete(HttpEntity(ContentTypes.`text/html(UTF-8
)`, "<h1>Say hello to akka-http</h1>"))
        }
      }
```

- Web API Design
  - eBook by Brian Mulloy
  - http://pages.apigee.com/web-api-design-ebook.html

Task 4

Develop a REST Service for your game using Akka HTTP.

Write your implementation as a further view layer (similar to the TUI).

This API will be used for performance testing later.

Also, introduce a REST API for your modules to prepare for an interprocess communication. This will be used to communicate between Docker instances.